

SIMATS ENGINEERING

ASSESSMENT TOOL 3

Mock Interview

COURSE NAME: COMPUTER VISION WITH OPENCV
COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: Understand video processing - motion computation - 3D vision - geometry. (BTL 6)

Assessment Tool Weightage: 10%

Mock Interview Questions

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.
2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.
3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.
4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

Image & Video Processing

5. A video stream drops frames during processing. Analyze the problem and redesign your approach to ensure smooth playback, explaining trade-offs and optimization techniques.
6. You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy.
7. A system must process both images and videos using the same pipeline. Propose a generalized design and explain how it handles both cases effectively.
8. You need to extract frames from a video and save only those with significant changes. Analyze the problem and design a method, explaining your decision criteria and approach.

Feature Detection & Drawing

9. You are asked to highlight key features in an image. Design a feature detection and visualization approach, explaining the techniques and their suitability.

10. A system must draw bounding boxes and labels for detected objects. Explain how you would structure this module, including reasoning behind chosen methods.
11. You need to compare two images and highlight differences visually. Analyze the problem and design a solution, justifying your approach.

Face & Expression Recognition

12. A face detection system produces multiple false positives. Analyze the issue and propose improvements, explaining how each change enhances accuracy.
13. You need to build a simple face recognition demo. Design a minimal working pipeline, explaining each stage and your design decisions.
14. A facial expression system fails with slight head movements. Analyze the limitation and propose modifications, justifying your approach.

OCR & Text Processing

15. You are given noisy scanned documents. Design preprocessing steps for OCR, explaining how each step improves recognition accuracy.
16. A system must detect text regions before recognition. Analyze the problem and propose a structured approach, justifying your design.

Surveillance & Applications

17. A surveillance system must highlight moving objects in real time. Design a lightweight solution, explaining your approach and optimization strategies.
18. You need to detect unusual activity in a video feed. Extend a basic motion detection system and justify your enhancements.
19. A system must count people entering a room using video. Design the counting logic and explain how you handle challenges such as occlusion and tracking.
20. You are asked to build a quick demo of a real-world Computer Vision application. Propose your idea, design the pipeline, and justify your choices clearly.

OpenCV Basics & Pipeline Thinking

21. You need to design an OpenCV pipeline that reads images from multiple formats (jpg, png, bmp) and processes them uniformly. Explain your design and justify how it ensures compatibility and efficiency.
22. A real-time application suffers from high latency during image display. Analyze the bottlenecks and redesign the pipeline for faster performance.
23. Design a modular OpenCV pipeline where preprocessing, feature detection, and visualization are separated. Justify how modularity improves maintainability.
24. You need to process images from a webcam and save selected frames. Design the pipeline and explain how you manage memory and performance.
25. A system must handle corrupted image files gracefully. Design error-handling strategies in your OpenCV pipeline and justify your approach.

Image & Video Processing

26. A video processing system must resize frames dynamically based on processing load. Design the approach and justify performance benefits.
27. You need to convert a color video into grayscale while preserving important details. Design the pipeline and justify your method.
28. A system must stabilize a shaky video feed. Propose a pipeline and justify your design choices.
29. You need to detect scene changes in a video. Design an efficient method and justify your decision criteria.
30. A system must compress video frames before processing. Design the approach and explain trade-offs.

Feature Detection & Drawing

31. You need to detect corners and edges simultaneously in an image. Design a combined pipeline and justify your approach.
32. A system must visualize feature matching between two images. Design the visualization pipeline and justify your choices.
33. You need to dynamically adjust feature detection thresholds. Design the logic and justify its effectiveness.
34. A system must track detected features across video frames. Design the approach and justify your tracking method.
35. You need to overlay detected keypoints on a video stream efficiently. Design the pipeline and justify performance considerations.

Face & Expression Recognition

36. A face detection system must run on low-power devices. Design an optimized pipeline and justify your model selection.
37. You need to handle multiple faces in a crowded scene. Design the detection and tracking pipeline and justify your approach.
38. A face recognition system must handle varying lighting conditions. Design preprocessing steps and justify their impact.
39. You need to integrate face detection with expression recognition. Design the pipeline and justify stage interactions.
40. A system must avoid false detections in background objects. Design improvements and justify your reasoning.

OCR & Text Processing

41. You need to extract text from real-time video frames. Design the pipeline and justify your approach.
42. A system must handle rotated text in images. Design preprocessing steps and justify their effectiveness.

43. You need to detect handwritten text in noisy images. Design a solution and justify your choices.
44. A system must segment text lines before OCR. Design the approach and justify its importance.
45. You need to improve OCR accuracy in low-resolution images. Design preprocessing techniques and justify their impact.

Surveillance & Applications

46. A system must detect intrusions in restricted zones. Design the pipeline and justify detection logic.
47. You need to build a system that detects abandoned objects in surveillance footage. Design the approach and justify your reasoning.
48. A surveillance system must operate under low-light conditions. Design enhancements and justify your approach.
49. You need to detect and track vehicles in traffic videos. Design the pipeline and justify each stage.
50. A system must identify suspicious motion patterns. Design the detection logic and justify your approach.

Advanced Pipeline Design

51. Design an OpenCV pipeline that integrates preprocessing, detection, tracking, and visualization. Justify the order of stages.
52. A system must handle both real-time and offline processing modes. Design a flexible pipeline and justify your approach.
53. You need to optimize memory usage in a video processing pipeline. Design strategies and justify their effectiveness.
54. A system must adapt dynamically to changing lighting and motion conditions. Design an adaptive pipeline and justify your logic.
55. You need to build a scalable OpenCV system for large datasets. Design the architecture and justify your choices.

Application-Oriented Design

56. Design a real-time attendance system using face recognition. Explain the pipeline and justify your design.
57. You need to build a vehicle counting system for highways. Design the pipeline and justify your approach.
58. Design a document scanning system that enhances readability automatically. Justify each stage of your pipeline.
59. You need to build a gesture recognition demo using OpenCV. Design the pipeline and justify your choices.
60. Design a smart monitoring system that combines motion detection and object recognition. Justify how each module contributes to system performance.

Code of Conduct:

*I Bhavya PS(192524017) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:**RUBRICS FOR CALCULATION:**

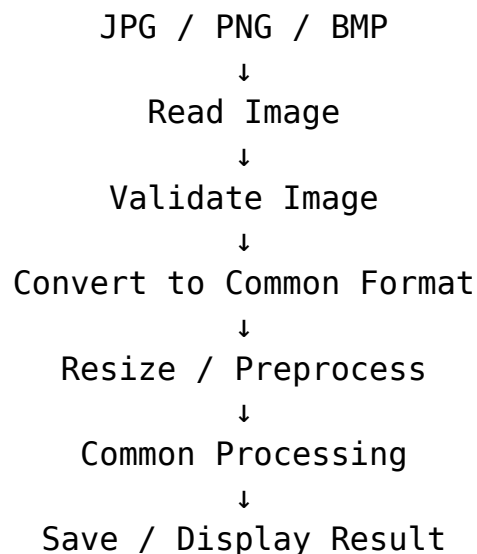
Criteria	Weight age	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. You need to design an OpenCV pipeline that reads images from multiple formats (JPG, PNG, BMP) and processes them uniformly. Explain your design and justify how it ensures compatibility and efficiency.

Answer: I would create a common input module that reads JPG, PNG, and BMP files using OpenCV's `imread()` function. After reading, all images are converted into a common format such as BGR and resized to a standard size before processing.

Pipeline:



OpenCV supports these common image formats, so the same processing functions can be applied after loading. I would also process images one at a time instead of keeping all images in memory.

Interview point: A common input format makes the pipeline compatible and reduces unnecessary format-specific processing.

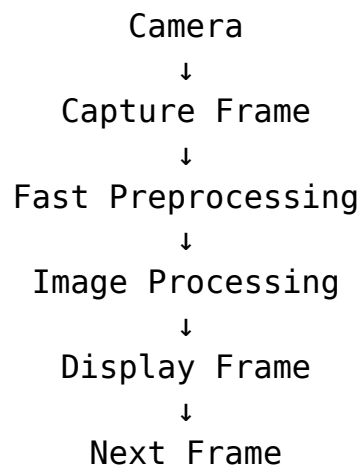
2. A real-time application suffers from high latency during image display. Analyze the bottlenecks and redesign the pipeline for faster performance.

Answer: I would first check whether the delay is caused by image processing, resizing, disk operations, or displaying frames. Frequent image saving and unnecessary conversions can also increase latency.

I would redesign the pipeline by:

- Processing only the required resolution.
- Avoiding unnecessary image conversions.
- Using efficient algorithms.
- Processing frames continuously instead of waiting for unnecessary operations.
- Using buffering or separate processing and display operations.
- Avoiding frequent disk writes.

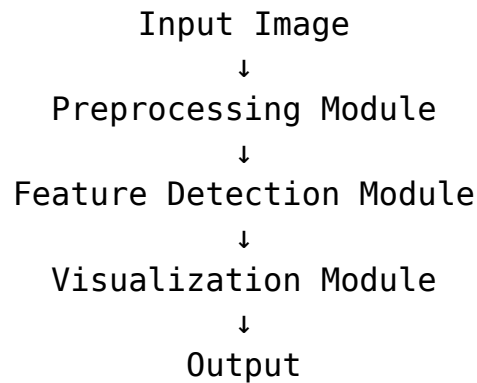
Improved pipeline:



Interview point: The main goal is to reduce the time spent on each frame so that processing keeps up with the incoming video.

3. Design a modular OpenCV pipeline where preprocessing, feature detection, and visualization are separated. Justify how modularity improves maintainability.

Answer: I would divide the application into three independent modules.



Modules:

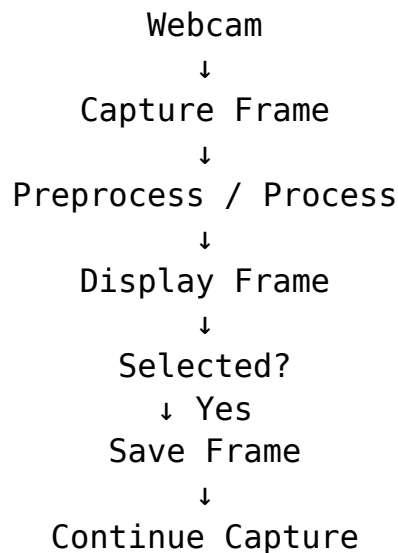
- Preprocessing: Resizing, grayscale conversion, noise removal, and contrast enhancement.
- Feature Detection: Detects edges, corners, keypoints, or other required features.
- Visualization: Draws detected features and displays or saves the result.

This design makes it easier to test, modify, and replace individual modules without changing the complete application.

Interview point: Modularity improves maintainability, testing, reusability, and debugging.

4. You need to process images from a webcam and save selected frames. Design the pipeline and explain how you manage memory and performance.

Answer: I would capture frames continuously using OpenCV's VideoCapture(). Instead of saving every frame, I would save only frames selected by the user or based on a specific condition.



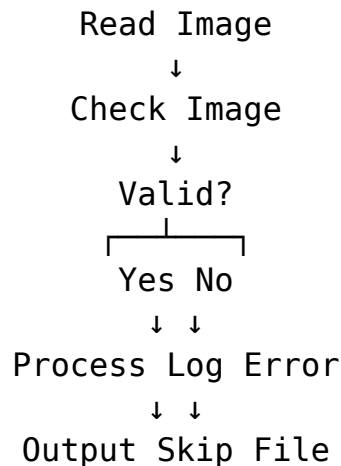
To manage memory, I would process one frame at a time and release resources after processing. I would also avoid storing unnecessary frames in a list.

For better performance, I would use an appropriate resolution and frame rate and avoid unnecessary processing.

Interview point: Processing frames one at a time prevents unnecessary memory usage while selective saving reduces disk operations.

5. A system must handle corrupted image files gracefully. Design error-handling strategies in your OpenCV pipeline and justify your approach.

Answer: I would validate every image immediately after reading it. In OpenCV, `imread()` can return an empty image when the file cannot be loaded correctly.



For corrupted files, I would:

- Check whether the image was successfully loaded.
- Catch file and processing errors.
- Record the filename and error in a log.
- Skip the corrupted file instead of stopping the complete pipeline.
- Continue processing the remaining valid images.
- Optionally move corrupted files to a separate folder for later inspection.

Interview point: Good error handling prevents one corrupted file from bringing down the entire image-processing system.